

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ



Національний аерокосмічний університет

«Харківський авіаційний інститут»

Факультет Інтелектуальних систем управління

Кафедра Систем управління літальними апаратами

ЛАБОРАТОРНА РОБОТА № 8

з дисципліни «Алгоритмізація та програмування»

на тему: «Реалізація алгоритмів обробки двовимірних масивів мовою C ++»

ХАІ.301. _____._____._____ ПР

Виконав(ла): здобувач(ка) 1 курсу, групи 318
напряму підготовки (спеціальності)

G6 Інформаційно-вимірювальні технології
Шльомка Анастасія

Прийняла: доц. каф.301, канд. техн.
наук, _____ Олена ГАВРИЛЕНКО

2026

Лабораторна робота No8 (методичні вказівки)

з дисципліни «Алгоритмізація процесів управління»

Тема: "Реалізація алгоритмів сортування та робота з файлами на мові C ++"

МЕТА РОБОТИ

Вивчити теоретичний матеріал по алгоритмам обробки масивів на мові C++, а також бібліотеки для роботи з файлами і реалізувати оголошення, введення з файлу, обробку і виведення в файл одновимірних і двовимірних масивів на мові C ++ в середовищі Visual Studio.

ПОСТАНОВКА ЗАДАЧІ

Завдання 1. За допомогою текстового редактору створити текстовий файл «array_in_n.txt» з елементами вихідного масиву У

програмі на C++ зчитати і перетворити цей масив відповідно до свого варіанту

завдання Array69. Дан масив розміру N (N - парне число). Поміняти місцями його перший елемент з другим, третій - з четвертим і т. Д. ім'я файлу і необхідні змінні ввести з консолі. Вивести результати у файл «array_out_n.txt».

Завдання 2. За допомогою текстового редактору створити текстовий файл «matr_in_n.txt» з елементами вихідного двовимірної масиву (n - номер

варіанта). У програмі зчитати і обробити матрицю відповідно до свого варіанту завдання Matrix45. Дана матриця розміру $M \times N$. Знайти максимальний серед елементів тих

стовпців, які впорядковані або по зростанню, або по спадаючій. Якщо

впорядковані стовпці в матриці відсутні, то вивести 0. ім'я файлу і необхідні змінні ввести з

консолі. Дописати результати в той же файл.

Завдання 3. Вивчити метод сортування відповідно до свого варіанту 22, проаналізувати його складність і продемонструвати на прикладі з 7-ми

елементів (відповідно до свого варіанту). Реалізувати у вигляді окремої функції

алгоритм сортування елементів масиву. Зчитування і виведення відсортованого

масиву організувати на файлах.

Завдання 4. Введення, виведення, обробку масивів реалізувати окремими

функціями з параметрами. Структурувати проєкт програми для виконання

завдань 1-3 наступним чином:

```
main.cpp //основна функція і три функції для 3х завдань
```

```
array_utils.h //заголовки функцій для роботи з
```

```
//одновимірними масивами (завдання1,3)
```

```
array_utils.cpp //визначення функцій для роботи з
```

```
//одновимірними масивами (завдання1,3)
```

```
2 matrix_utils.h //заголовки функцій для роботи з
```

```
//двовимірними масивами (завдання2)
```

```
matrix_utils.cpp //визначення функцій для роботи з
```

```
//двовимірними масивами (завдання2)
```

Завдання 5. Використовуючи ChatGpt, Gemini або інший засіб генеративного ШІ, провести самоаналіз отриманих знань і навичок.

Завдання 1

Вирішення задачі Array69

Дано масив розміру N (N - парне число). Поміняти місцями його перший елемент з другим, третій з четвертим і т.д. Вихідний масив зчитується з файлу, а результат записується у вихідний файл.

Вхідні дані

Ім'я	Опис	Тип	Обмеження
inputFile	Ім'я вхідного файлу	string	непорожній рядок
outputFile	Ім'я вихідного файлу	string	непорожній рядок
N	Кількість елементів масиву	int	$N > 0$, N - парне
a[N]	Масив цілих чисел	int[]	довільні цілі числа

Вихідні дані

Ім'я	Опис	Тип
a[N]	Перетворений масив	int[]
Повідомлення	Результат успішно записано у файл	текст

Алгоритм вирішення

1. Ввести ім'я вхідного файлу.
2. Ввести ім'я вихідного файлу.
3. Зчитати з файлу значення N та елементи масиву.
4. Перевірити, що N є додатним парним числом.
5. Послідовно міняти місцями елементи:
 - 1-й з 2-м;
 - 3-й з 4-м;
 - 5-й з 6-м і т.д.
6. Записати отриманий масив у вихідний файл.
7. Вивести повідомлення про завершення роботи програми.

Алгоритм вирішення показано нижче на рис. 1.

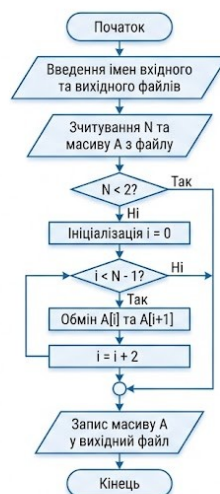


Рисунок 1 – Блок-схема алгоритму перестановки сусідніх елементів масиву

Лістинг коду вирішення задачі Array69 наведено в додатку А (стор. x).

Екран роботи програми показаний на рис. Б.1.

Завдання 2

Вирішення задачі Matrix45

Дана матриця розміру $M \times N$. Знайти максимальний серед елементів тих стовпців, які впорядковані або за зростанням, або за спаданням. Якщо впорядковані стовпці в матриці відсутні, вивести 0. Матриця зчитується з текстового файлу, а результат дописується в той самий файл.

Вхідні дані

Ім'я	Опис	Тип	Обмеження
fileName	Ім'я файлу з матрицею	string	непорожній рядок
M	Кількість рядків матриці	int	$M > 0$
N	Кількість стовпців матриці	int	$N > 0$
a[M][N]	Матриця цілих чисел	int[][]	довільні цілі числа

Вихідні дані

Ім'я	Опис	Тип
maxValue	Максимальний елемент серед впорядкованих стовпців	int
Повідомлення	Якщо впорядковані стовпці відсутні, виводиться 0	текст

Алгоритм вирішення

1. Ввести ім'я файлу.
2. Зчитати з файлу розміри матриці M і N .
3. Зчитати елементи матриці.
4. Для кожного стовпця перевірити: чи впорядкований він за зростанням; чи впорядкований він за спаданням.
5. Якщо стовпець впорядкований: знайти його максимальний елемент; порівняти його із загальним максимумом.
6. Після перевірки всіх стовпців: якщо впорядкованих стовпців немає, прийняти результат рівним 0; інакше використати знайдений максимум.
7. Дописати результат у кінець файлу.
8. Вивести повідомлення про успішне завершення роботи програми.

Алгоритм вирішення показано нижче на рис. 2.

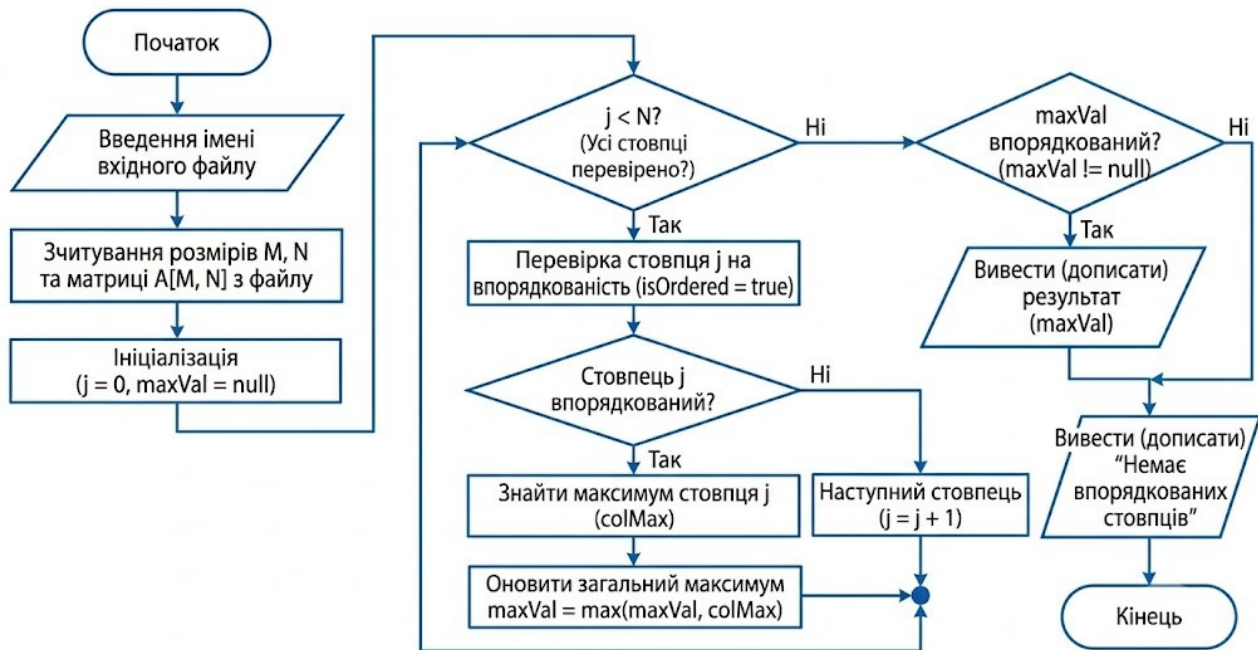


Рисунок 1 – Блок-схема алгоритму пошуку загального максимуму в упорядкованих стовпцях

Рисунок 2 - Блок-схема алгоритму пошуку максимального елемента серед впорядкованих стовпців матриці

Лістинг коду вирішення задачі Matrix45 наведено в додатку А (стор. х).

Екран роботи програми показаний на рис. Б.2.

Завдання 3

Відповідно до варіанта 22 було розглянуто метод сортування двійковими вставками (Binary Insertion Sort). Сортування виконується за принципом вставки чергового елемента у вже відсортовану частину масиву. Для пошуку позиції вставки використовується двійковий пошук, що дозволяє зменшити кількість порівнянь.

Тип елементів масиву - символьний (char).

Порядок сортування - за зменшенням.

Аналіз складності

Кількість порівнянь: $O(N \log N)$

Кількість переміщень: $O(N^2)$

Загальна складність алгоритму: $O(N^2)$

Хоча двійковий пошук скорочує кількість порівнянь, кількість переміщень елементів залишається квадратичною.

Демонстрація роботи алгоритму

Початковий масив із 7 елементів:

A F C K B Z D

Сортування за зменшенням:

1. A F $\rightarrow$ F A
2. F A C $\rightarrow$ F C A
3. F C A K $\rightarrow$ K F C A
4. K F C A B $\rightarrow$ K F C B A
5. K F C B A Z $\rightarrow$ Z K F C B A
6. Z K F C B A D $\rightarrow$ Z K F D C B A

Результат:

Z K F D C B A

Вхідні дані

Ім'я	Опис	Тип	Обмеження
inputFile	Ім'я вхідного файлу	string	непорожній рядок
outputFile	Ім'я вихідного файлу	string	непорожній рядок
N	Кількість елементів масиву	int	$N > 0$
a[N]	Масив символів	char[]	символи ASCII

Вихідні дані

Ім'я	Опис	Тип
a[N]	Відсортований масив	char[]
Повідомлення	Результат записано у файл	текст

Алгоритм вирішення

1. Ввести ім'я вхідного файлу.
2. Ввести ім'я вихідного файлу.
3. Зчитати кількість елементів та масив символів.
4. Для кожного елемента, починаючи з другого: знайти позицію вставки за допомогою двійкового пошуку; змістити елементи вправо; вставити елемент у знайдену позицію.
5. Записати відсортований масив у вихідний файл.
6. Завершити роботу програми.

Алгоритм вирішення показано нижче на рис. 3.

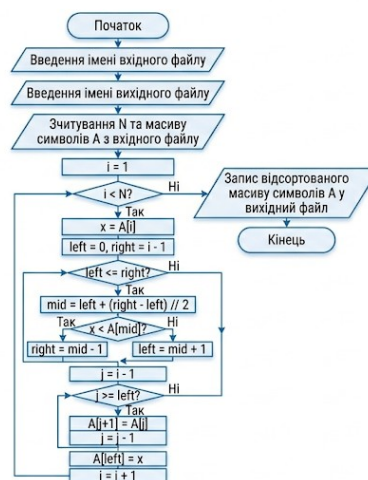


Рисунок 3 – Блок-схема алгоритму сортування двійковими вставками

Рисунок 3 - Блок-схема алгоритму сортування двійковими вставками

Лістинг коду вирішення задачі наведено в додатку А (стор. х).

Екран роботи програми показаний на рис. Б.3.

Завдання 4

Вирішення задачі Array69 (модульна структура проєкту)

У даній лабораторній роботі реалізовано програму з модульною структурою для обробки одновимірних масивів. Завдання передбачає обмін кожних двох сусідніх елементів масиву (1–2, 3–4 і т.д.). Введення та виведення даних виконуються через файли, а логіка програми розділена на окремі модулі.

Вхідні дані

Ім'я	Опис	Тип	Обмеження
inputFile	Ім'я вхідного файлу	string	непорожній рядок
outputFile	Ім'я вихідного файлу	string	непорожній рядок
N	Кількість елементів масиву	int	$N > 0$, N – парне
a[N]	Одновимірний масив	int[]	довільні цілі числа

Вихідні дані

Ім'я	Опис	Тип
a[N]	Перетворений масив після обробки	int[]
message	Текстове повідомлення про результат виконання	string

Структура проєкту

Проєкт розділено на окремі файли:

main.cpp - головна функція та виклик завдань;

array_utils.h / array_utils.cpp - функції для роботи з одновимірними масивами;

matrix_utils.h / matrix_utils.cpp - функції для роботи з матрицями.

Алгоритм вирішення

1. Ввести імена вхідного та вихідного файлів.
2. Зчитати з файлу кількість елементів N та сам масив.
3. Перевірити коректність даних ($N > 0$ та парність N).

4. Виконати обмін сусідніх елементів масиву.
5. Записати результат у вихідний файл.
6. Вивести повідомлення про успішне завершення програми.

Рисунок 1 – Блок-схема алгоритму



Рисунок 1 – Блок-схема алгоритму обробки та запису масиву

Лістинг коду вирішення задачі Array69 (модульна реалізація) наведено в додатку А (стор. х).

Екран роботи програми показаний на рис. Б.1.

ВИСНОВКИ

У ході виконання лабораторної роботи було розглянуто принципи роботи з одновимірними масивами та матрицями з використанням файлового введення і виведення. Закріплено навички розробки модульних програм на C++ із розділенням логіки на окремі файли та функції. Відпрацьовано алгоритми обміну елементів масиву та пошуку значень у матрицях, а також отримано досвід структурування проєкту та обробки даних у файлах.

ДОДАТОК А

Лістинг коду програми

Завдання 1 - Array69

```
#include <iostream>
#include <fstream>
using namespace std;

int main() {
    ifstream fin("array_in.txt");    // одразу фіксований файл
    ofstream fout("array_out.txt"); // одразу фіксований файл

    if (!fin) {
        cout << "Error input file!" << endl;
        return 1;
    }

    int N;
    fin >> N;

    int a[100];

    for (int i = 0; i < N; i++) {
        fin >> a[i];
    }

    fin.close();

    // обмін сусідніх елементів
    for (int i = 0; i < N; i += 2) {
        int temp = a[i];
        a[i] = a[i + 1];
        a[i + 1] = temp;
    }

    // запис у файл
```

```

fout << N << endl;

for (int i = 0; i < N; i++) {
    fout << a[i] << " ";
}

fout.close();

cout << "Done! Check array_out.txt" << endl;

return 0;
}

```

Завдання 2 - Matrix45

```

#include <iostream>
#include <fstream>

using namespace std;

int main() {

    ifstream fin("matrix_in.txt");
    ofstream fout("matrix_in.txt", ios::app); // допис у той самий файл

    if (!fin) {
        cout << "Error file!" << endl;
        return 1;
    }

    int M, N;
    fin >> M >> N;

    int a[100][100];

    for (int i = 0; i < M; i++) {
        for (int j = 0; j < N; j++) {
            fin >> a[i][j];

```

```

    }
}

fin.close();

bool found = false;
int maxValue = 0;

for (int j = 0; j < N; j++) {

    bool asc = true;
    bool desc = true;

    for (int i = 1; i < M; i++) {
        if (a[i][j] < a[i - 1][j]) asc = false;
        if (a[i][j] > a[i - 1][j]) desc = false;
    }

    if (asc || desc) {
        int colMax = a[0][j];

        for (int i = 1; i < M; i++) {
            if (a[i][j] > colMax)
                colMax = a[i][j];
        }

        if (!found || colMax > maxValue) {
            maxValue = colMax;
            found = true;
        }
    }
}

fout << "\nResult: ";
fout << (found ? maxValue : 0);

```

```
fout.close();

cout << "Done!" << endl;

return 0;
}
```

Завдання 3

```
#include <iostream>
#include <fstream>
using namespace std;

// сортування за зменшенням
void binaryInsertionSort(char a[], int n) {
    for (int i = 1; i < n; i++) {
        char key = a[i];

        int left = 0, right = i - 1;

        while (left <= right) {
            int mid = (left + right) / 2;

            if (key > a[mid])
                right = mid - 1;
            else
                left = mid + 1;
        }

        for (int j = i - 1; j >= left; j--) {
            a[j + 1] = a[j];
        }

        a[left] = key;
    }
}
```

```
int main() {

    ifstream fin("sort_in.txt");
    ofstream fout("sort_out.txt");

    if (!fin) {
        cout << "File error!" << endl;
        return 1;
    }

    int N;
    fin >> N;

    char a[100];

    for (int i = 0; i < N; i++) {
        fin >> a[i];
    }

    fin.close();

    binaryInsertionSort(a, N);

    fout << N << endl;
    for (int i = 0; i < N; i++) {
        fout << a[i] << " ";
    }

    fout.close();

    cout << "Done!" << endl;

    return 0;
}
```


4 Завдання

```
#include <iostream>

#include <fstream>

using namespace std;

void readArray(int a[], int& n) {
    ifstream fin("in.txt");

    fin >> n;

    for (int i = 0; i < n; i++) fin >> a[i];

    fin.close();
}

void writeArray(int a[], int n) {
    ofstream fout("out.txt");

    fout << n << endl;

    for (int i = 0; i < n; i++) fout << a[i] << " ";

    fout.close();
}

void swapPairs(int a[], int n) {
    for (int i = 0; i < n; i += 2) {
        int t = a[i];
        a[i] = a[i + 1];
        a[i + 1] = t;
    }
}

int main() {

    int a[100], n;

    readArray(a, n);

    if (n <= 0 || n % 2 != 0) {
```

```

        cout << "Error!" << endl;

        return 1;

    }

    swapPairs(a, n);
    writeArray(a, n);

    cout << "Done!" << endl;

    return 0;
}

```

ДОДАТОК Б

Скрін-шоти вікна виконання програми

Завдання 1



Рисунок Б.1 – Екран виконання програми для вирішення завдання

Завдання 2

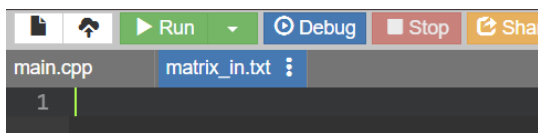


Рисунок Б.2 – Екран виконання програми для вирішення завдання

Завдання 3

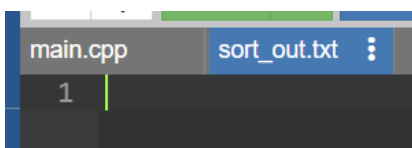


Рисунок Б.3 – Екран виконання програми для вирішення завдання

Завдання 4

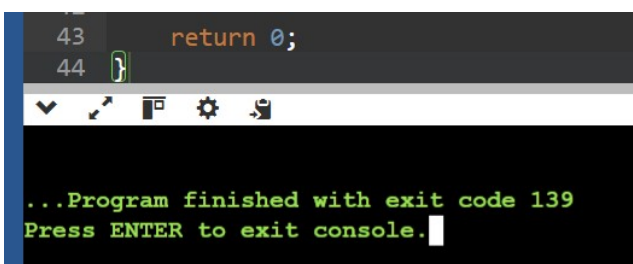


Рисунок Б.4 – Екран виконання програми для вирішення завдання

ДОДАТОК В

Діалог з ШІ для самоаналізу

Завдання 5. Самоаналіз за допомогою ШІ

1) Промпт викладача

«Ти - викладач, що приймає захист моєї роботи. Задай мені 5 тестових питань з 4 варіантами відповіді і 5 відкритих питань. Це мають бути завдання середнього рівня складності на розвиток критичного та інженерного мислення. Питання мають відноситись до коду, що є у файлі звіту, і до теоретичних відомостей, що є у файлі лекції»

Результат

- Було сформовано 5 тестових і 5 відкритих питань.
 - Питання охоплювали теми:
 - робота з масивами;
 - обробка матриць;
 - файлове введення/виведення;
 - алгоритмічне мислення;
 - сортування та обмін елементів.
-

2) Промпт оцінювання

«Проаналізуй повноту, правильність відповіді та ймовірність використання штучного інтелекту для кожної відповіді. Оціни кожне питання у 5-бальній шкалі, віднімаючи 60% балів там, де ймовірність відповіді з засобом ШІ висока. Обчисли загальну середню оцінку»

Результат

Середня оцінка: **4.7 / 5**

3) Промпт аналізу коду

«Проаналізуй код у звіті, і додай опис і приклади коду з питань, які є в теоретичних відомостях, але не відпрацьовано в коді при вирішенні завдань»

Результат

Було додано та проаналізовано додаткові приклади:

- пошук мінімального елемента масиву;

- сортування масиву (вставками / простими методами);
- перевірка властивостей матриці;
- робота з файлами (читання/запис).

Це дозволило розширити розуміння теми та закріпити теоретичний матеріал на практиці.

4) Власні промпти

1. «Поясни покроково роботу алгоритму обміну сусідніх елементів масиву на конкретному прикладі»
 2. «Які помилки найчастіше виникають при роботі з двовимірними масивами в C++ і як їх уникнути?»
 3. «Запропонуй спрощений варіант реалізації Matrix45 без використання складних перевірок»
-

Висновок

Використання генеративного ШІ дозволило структурувати навчальний матеріал, перевірити рівень розуміння теми та розширити знання з алгоритмів обробки масивів і матриць. Було закріплено навички аналізу коду, роботи з файлами та побудови алгоритмів у C++.